

→ **Polymorphism** :  
 Polymorphism is one of the **four pillar of OOP**.  
 The words **polymorphism** comes from:

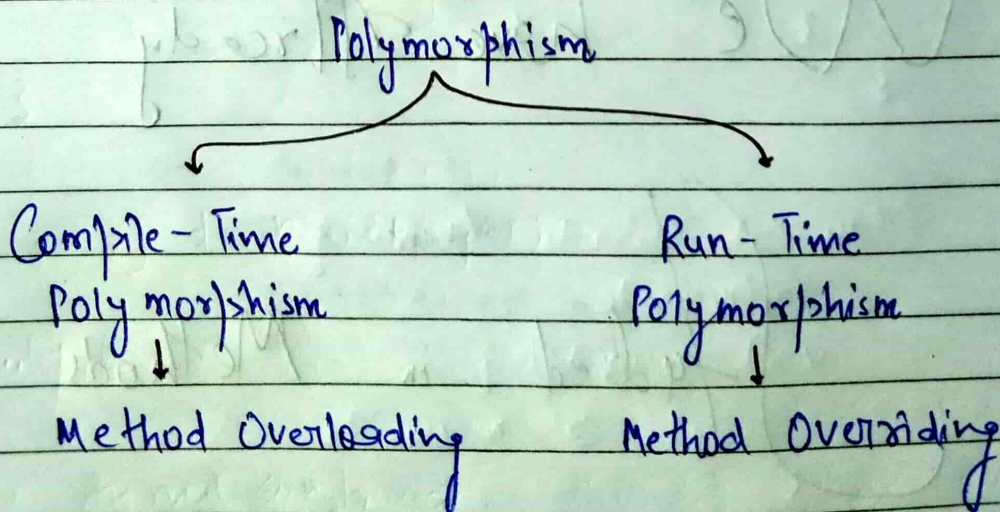
- **Poly** - Many
- **Morphism** - forms

So,

**Polymorphism** means **one thing** having **many forms**.

In java, **Polymorphism** allows the **same method name** or **reference** to **behave differently** in **different situation**.

→ **Types of Polymorphism** :  
 Java mainly supports two types:



1. Compile-Time Polymorphism:  
 Compile-time polymorphism is also called:

- Static polymorphism
- Early binding
- Method overloading

The compiler decides which method should be called at compile time.

Method Overloading:

Method Overloading means having multiple method with the same name but different parameter list.

We have Already

Studied in Methods in  
 Java { Method Overloading }

## 2.7 Run-Time Polymorphism:

Run-time polymorphism is also called:

- Dynamic Polymorphism
- Late binding
- Dynamic method dispatch

It is achieved through:

### Method Overriding

In runtime polymorphism, the method that executes is determined **at runtime**, based on the actual object.

#### → Method Overriding:

When a child class provides its own implementation of a method that is already **define** in its **parent class**, it is called **method overriding**.

Eg:

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
```

```
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

here,

Animal → Sound()

Dog → Sound()

The child ~~dog~~ Dog Overrides the parent's Sound() method.

Eg: Run-time Polymorphism:

```
Class Animal {
    void sound() {
        Sout("Animal Sound");
    }
}
```

```
Class Dog extends Animal {
    @Override
    void sound() {
        Sout("Dog barks");
    }
}
```

```
Class Cat extends Animal {
    @Override
    void sound() {
        Sout("Cat meows");
    }
}
```

```
Class pogo {
```

```
psvm {  
    Animal a;  
  
    a = new Dog();  
    a.sound();  
  
    a = new Cat();  
    a.sound();  
}  
}
```

output :  
Dog barks  
Cat meows

→ Important concept :

Animal a = new Dog();  
          ↑                  ↑  
Reference Type      Actual Object type

So :

Reference = Animal  
Object = Object

When we call :  
a.sound()

Java looks at the **Actual Object** !  
new Dog();

Dog.Sound()  
 is executed.

→ @Override Annotation:

When overriding a method, it is good practice to use:

@Override

It tells the compiler:

"I intend to override a parent method."

If we accidentally make a mistake in the method signature, the compiler can catch it.

**Akash**